

The Frame Problem

Video Lesson

Duration: 00:12:06

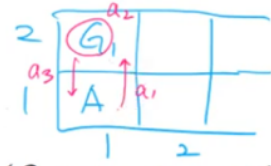
Slide Deck

M10S03 (<https://canvas.tamu.edu/courses/430127/files/92570685?wrap=1>) 

Lesson Transcript

In this lecture, we look at the frame problem. The frame problem arises in situation calculus because it is difficult to track what changes and what does not change as a consequence of taking an action.

Frame Problem



- In the previous slide, we cannot deduce if the following can be proven (G_1 represents a particular lump of gold):

$$At(G_1, [1, 1], Result([Go([1, 1], [1, 2]), Grab(G_1), Go([1, 2], [1, 1])], S_0))$$
- It is because the effect axioms say only *what should change*, but not *what does not change* when actions are taken (Gold may have moved when the agent moved).
- Initial solution: *Frame axioms*

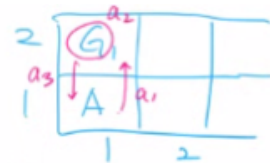
$$At(o, x, s) \wedge (o \neq Agent) \wedge \neg Holding(o, s) \\ \rightarrow At(o, x, Result(Go(y, z), s)).$$

This says moving does not affect the gold when it is not held. Problem is that you need $O(AF)$ such axioms for all (action, fluent) pair (A: num of actions, F: num of fluent predicates).

In the previous slide, we looked at the situation where the agent is initially at location [1, 1], and the goal object G_1 is at location [1, 2]. The problem in situation calculus is, given this situation, we cannot deduce if this following sequence of actions and the resulting fluent would be true.

We start from the situation at zero and take three actions, a_1 , a_2 , and a_3 . a_1 is go from [1, 1] to [1, 2]. So, that corresponds to this move a_1 . The second action is to grab G_1 , and then the third action is to go from [1, 2] to [2, 1].

Frame Problem



- In the previous slide, we cannot deduce if the following can be proven (G_1 represents a particular lump of gold):

$$At(G_1, [1, 1], Result([Go([1, 1], [1, 2]), Grab(G_1), Go([1, 2], [1, 1])], S_0))$$
- It is because the effect axioms say only *what should change*, but not *what does not change* when actions are taken (Gold may have moved when the agent moved).
- Initial solution: *Frame axioms*



$$At(o, x, s) \wedge (o \neq Agent) \wedge \neg Holding(o, s) \\ \rightarrow At(o, x, Result(Go(y, z), s)).$$

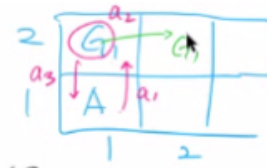
This says moving does not affect the gold when it is not held. Problem is that you need $O(AF)$ such axioms for all (action, fluent) pair (A: num of actions, F: num of fluent predicates).

To illustrate this, we start from S_0 . We take action 1. We end up in situation 1. We take action 2, and we end up in situation 2.

We take action 3, and we end up in situation 3. So, the result of taking these three actions, starting from situation 0, would be situation 3. So, we want to know whether object G_1 is at location $[1, 1]$.

Normally this would be true because when the agent moves to this location and grabs the object, G_1 , then when the agent comes back because the agent is holding it, that object, G_1 , will now be relocated to location $[1, 1]$. Hence, this predicate or the fluent would now be true.

Frame Problem



- In the previous slide, we cannot deduce if the following can be proven (G_1 represents a particular lump of gold):

$At(G_1, [1, 1], \text{Result}([Go([1, 1], [1, 2]), \text{Grab}(G_1), Go([1, 2], [1, 1])], S_0))$

- It is because the effect axioms say only *what should change*, but not *what does not change when actions are taken* (Gold may have moved when the agent moved).
- Initial solution: *Frame axioms*



$$At(o, x, s) \wedge (o \neq \text{Agent}) \wedge \neg \text{Holding}(o, s)$$

$$\rightarrow At(o, x, \text{Result}(Go(y, z), s)).$$

This says moving does not affect the gold when it is not held. Problem is that you need $O(AF)$ such axioms for all (action, fluent) pair (A : num of actions, F : num of fluent predicates).

However, this may or may not be true depending on the behavior of the object G_1 . While the agent was taking this action, a_1 , if this object moved away from this location to some other place, then perhaps it may not be possible to grab it and bring it back.

The reason why we cannot deduce whether object G_1 is now back in location $[1, 1]$ after taking these three actions is that they affect axioms. They only say what should change but not what does not change when actions are taken.

Frame Problem

- In the previous slide, we cannot deduce if the following can be proven (G_1 represents a particular lump of gold):

$$At(G_1, [1, 1], Result([Go([1, 1], [1, 2]), Grab(G_1), Go([1, 2], [1, 1])), S_0)$$

- It is because the effect axioms say only *what should change*, but not *what does not change when actions are taken* (Gold may have moved when the agent moved).
- Initial solution: *Frame axioms*

$$At(o, x, s) \wedge (o \neq Agent) \wedge \neg Holding(o, s) \\ \rightarrow At(o, x, Result(Go(y, z), s)).$$

This says moving does not affect the gold when it is not held. Problem is that you need $O(AF)$ such axioms for all (*action, fluent*) pair (A : num of actions, F : num of fluent predicates).

An initial solution to this problem is introducing the frame axiom. Let us look at this. If object o is at location x in situation s , and if the object is not the agent, and if it happens that the agent is not holding onto the object in the situation s , then the object remains in location x when the agent moves from some location y to some other location z starting from this situation s .

This basically says that moving does not affect the gold when it is not held by the agent. The problem is that we need eight times F many such axioms to describe all of these contingencies where a is the number of possible actions, and F is the number of all of these fluents in the domain.

Two Frame Problems

- Representational frame problem:
Explained in the previous slide
- Inferential frame problem:
Projection of results of a t -step sequence of actions in time $O(Et)$ (E is the number of effects, typically much less than F , the number of fluent predicates), rather than $O(Ft)$ or $O(AEt)$.

There are two frame problems. The first is the representation frame problem explained in the previous slide, and the next one is the inferential frame problem. The inferential frame problem arises because we want to project the results of a t -step sequence of actions in time on the order of E times t , where E is the number of effects, and t is the number of steps.

If we use a brute force approach, we have to check all of the fluents for each time step t . So, this

whole thing will become a process that takes an order of F times t , but we want to make it much more effective by only looking at things that have a direct effect based on the action at each step.

Solving the Representational Frame Problem

- Consider how each fluent predicate evolves over time: Successor-state axioms
 $\text{Action is possible} \rightarrow$
 $(\text{Fluent is true in result state} \leftrightarrow (1) \text{ Action's effect made it true} \vee (2) \text{ It was true before and action left it alone}).$
- Example:
 $\text{Poss}(a, s) \rightarrow$
 $(\text{At}(\text{Agent}, y, \text{Result}(a, s)) \leftrightarrow a = \text{Go}(x, y) \vee (\text{At}(\text{Agent}, y, s) \wedge a \neq \text{Go}(y, z))).$
- Remaining issues: implicit effect (moving while holding something moves that something as well)
– ramification problem. Can solve by using a more general successor-state axiom.

To solve the representational frame problem, we introduced the concept of successor-state axioms. What we are interested in is whether a particular fluent is true in the resulting state of taking a particular action.

Here, the successor-state axiom says when the axiom is possible, then a particular fluent is true in the resulting state if and only if the effect of the actions made it true. So, previously the fluent was false, but because of the action, it became true, or it was true already before the action, and the action did not affect it.

For example, let us consider this fluent. Is the agent at location y due to a particular action taken in situation s ? So, the first possibility, the action's effect made it true, would be when the action a was go from x to y . And here it was possible. So, this action will be taken. As a result, y would be the new location of the agent, and hence this, predicate, or this fluent would evaluate to true. So, that is the first case.

The second case is it was true before, and the action left it alone. Let us take a look at this. The agent was already at location y in situation s . So, this fluent is already true, and the action was not go from y to some other location z .

There are, of course, remaining issues. For example, some kind of implicit effect while moving and holding onto something. This would actually allow the particular object that is being held also to move. This is called the ramification problem.

And we can solve this by using a more general successor-state axiom.



Solving the Inferential Frame Problem

- Given a t -step plan p ($S_t = \text{Result}(p, S_0)$), decide which fluents are true in S_t .
- We need to consider each of the F frame axioms for each time step t .
- Axioms have an average size of AE/F , we have an $O(AEt)$ inferential work. Most of the work is done copying unchanged fluents from time step to time step.
- Solutions: use fluent calculus rather than situation calculus, or make the process more efficient.

How about the inferential frame problem? Here, we want to know, given a t -step plan p , when we start from situation S_0 and take this plan, then in the resulting situation, S_t , we want to decide which fluents are true. As we briefly mentioned earlier, if we take t steps and if we want to be thorough, we have to investigate whether all F of these frame axioms validate to be true.

That is, the computational work is on the order of F times t . But each axiom here has an average size of AE times F , where A is the number of actions, E is the average number of effects, and F is the number of fluents. So, when we multiply these, then we have the order of AE times t inferential work. The problem is that this is very inefficient, and most of the work is done just copying unchanged fluents from time step to time step.

The solution is to use something called "fluent calculus" rather than "situation calculus" to make this process more efficient.

Solving the Inferential Frame Problem

- Typical frame axiom:

$$Poss(a, s) \rightarrow [F_i(Result(a, s)) \leftrightarrow \{(a = A_1 \vee a = A_2 \dots) \vee (F_i(s) \wedge (a \neq A_3) \wedge (a \neq A_4) \dots)\}] \quad (1)$$

- Several actions make the fluent true and several make the fluent false.
→ Formalize this using the predicates $PosEffect(a, F_i)$ and $NegEffect(a, F_i)$.

$$Poss(a, s) \rightarrow [F_i(Result(a, s)) \leftrightarrow \{PosEffect(a, F_i) \vee (F_i(s) \wedge \neg NegEffect(a, F_i))\}] \quad (2)$$

$PosEffect(A_1, F_i), PosEffect(A_2, F_i)$

$NegEffect(A_3, F_i), NegEffect(A_4, F_i)$

- * This can be done efficiently: get current action, and fetch its effects, then update those fluents $O(Et)$.

Let us consider one approach shown here where we define a new predicate, positive effect, and negative effect. Here is a typical frame axiom. If an action a is possible in situation s , then a particular fluent F_i holds in this resulting situation if the action made it true or this fluent was already true, and the action did not change it.

We can abstract these actions and these actions in relation to this particular fluent F_i into positive effect a, F_i and negative effect a, F_i . So, this means that this fluent F_i , when an action a is taken, will result in true. That means this action a has a positive effect on this fluent F_i . On the other hand, a negative effect means this action a has a negative effect on this fluent F_i . Meaning that this fluent F_i will turn false when this action is taken.

With that, we can map this part, the positive effect part, for all of the actions. That makes this fluent to be true into a single predicate here, positive effect a, F_i , and for the second part, where the fluent F_i was already true, and the action was none of these, we again extract all of that into negative effect so that this fluent i being true previously in the initial situation s and we can find the conjunction of that with a negation of the negative effect, meaning that we do not want this to happen, meaning that action a changing fluent F_i . And since we put this negation on that, we will exclude all of the actions.

So, along with this particular rule that is rewritten from here, we have to add more facts that action 1 here is a positive effect on fluent F_i and Action 2 is also a positive effect on fluent F_i , whereas A_3 , action 3, and action 4 are negative effects on fluent F_i .

So, if we do this, this can be done efficiently. So, we get the current action and then fetch its effects, then update those fluents that are affected in time on the order of E times t , where E is the number of effects on average.

In the next lecture, which will be the last lecture in this module, we will look at other forms of planning and also look at an interesting demo.

CSCE
625

MODULE 10 | FRAME THE PROBLEM